Programmation Java

Chapitre 2 : Classes et Objets

Saoudi Haythem

Polytechnique Sousse

2025-2026

Plan

1. Introduction aux Classes et Objets
2. Déclaration d'une Classe
3. Constructeurs
4. Instanciation et Utilisation
5. Attributs et Méthodes Static
6. Packages et Imports
7. La Classe Scanner
8. La Classe String
9. Les Associations
10. QCM
11. Exercices

Programmation Orientée Objet (POO)

Définition

La POO est un paradigme de programmation qui organise le code autour d'**objets** plutôt que de fonctions et de logique.

➤ **Concepts clés :**

- Classe
- Objet
- Attributs
- Méthodes
- Encapsulation

★ **Avantages :**

- Réutilisabilité
- Modularité
- Maintenance facile
- Abstraction

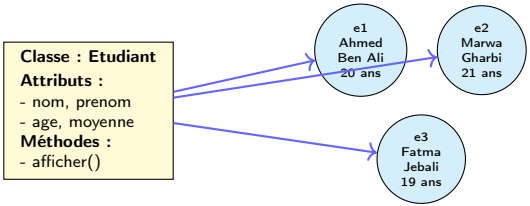
Classe vs Objet

★ Classe

- **Modèle** ou **plan**
- Définit la structure
- Décrit les attributs et méthodes
- Comme un *moule*

● Objet

- **Instance** d'une classe
- Entité concrète en mémoire
- Possède des valeurs propres
- Comme un *gâteau* fait avec le moule



Structure d'une Classe

```java
public class NomClasse {
    // Attributs (variables d'instance)
    private type attribut1;
    private type attribut2;

    // Constructeur
    public NomClasse(parametres) {
        // initialisation
    }

    // Methodes
    public type nomMethode(parametres) {
        // corps de la methode
    }
}
```

Exemple : Classe Etudiant

```java
public class Etudiant {
    // Attributs
    private String nom;
    private String prenom;
    private int age;
    private double moyenne;

    // Constructeur
    public Etudiant(String nom, String prenom, int age, double moyenne) {
        this.nom = nom;
        this.prenom = prenom;
        this.age = age;
        this.moyenne = moyenne;
    }

    // Methode pour afficher les infos
    public void afficher() {
        System.out.println(prenom + " " + nom + " - Age: " + age);
    }
}
```

Les Modificateurs d'Accès

Modificateur	Classe	Package	Sous-classe	Monde
`public`	✓	✓	✓	✓
`protected`	✓	✓	✓	
(default)	✓	✓		
`private`	✓			

Règle importante

Les **attributs** doivent être `private` et accessibles via des **getters/setters**.

L'Encapsulation et les Accesseurs

Principe de l'Encapsulation

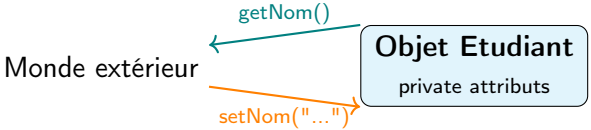
L'encapsulation consiste à **protéger les données** d'un objet en les rendant `private` et en contrôlant leur accès via des méthodes publiques.

⇓ **Getter (Accesseur) :**

- Permet de **lire** la valeur d'un attribut
- Nom : `getAttribut()`
- Pour boolean : `isAttribut()`

⇑ **Setter (Mutateur) :**

- Permet de **modifier** la valeur d'un attribut
- Nom : `setAttribut(valeur)`
- Peut inclure une **validation**



Exemple : Getters et Setters pour Etudiant

```java
public class Etudiant {
    private String nom;
    private int age;
    private double moyenne;
    // Constructeur
    public Etudiant(String nom, int age, double moyenne) {
        this.nom = nom;
        this.age = age;
        this.moyenne = moyenne;
    }
    // Getter pour nom
    public String getNom() { return nom; }
    // Setter pour nom
    public void setNom(String nom) { this.nom = nom; }
    // Getter pour age
    public int getAge() { return age; }
    // Setter pour age
    public void setAge(int age) { this.age = age; }
    // Getter pour moyenne
    public double getMoyenne() { return moyenne; }
    // Setter pour moyenne
    public void setMoyenne(double moyenne) { this.moyenne = moyenne; }
}
```

Validation dans les Setters

Avantage clé

Les setters permettent de **valider les données** avant de les affecter aux attributs.

```java
public class Etudiant {
    private int age;
    private double moyenne;
    // Setter avec validation pour age
    public void setAge(int age) {
        if (age > 0 && age < 120) {
            this.age = age;
        } else {
            System.out.println("Age invalide !");
        }
    }
    // Setter avec validation pour moyenne
    public void setMoyenne(double moyenne) {
        if (moyenne >= 0 && moyenne <= 20) {
            this.moyenne = moyenne;
        } else {
            System.out.println("Moyenne doit etre entre 0 et 20");
        }
    }
}
```

Récapitulatif : Getters et Setters

Type	Getter	Setter
`String nom`	`String getNom()`	`void setNom(String nom)`
`int age`	`int getAge()`	`void setAge(int age)`
`double moyenne`	`double getMoyenne()`	`void setMoyenne(double m)`
`boolean actif`	`boolean isActif()`	`void setActif(boolean a)`

✓ Bonnes pratiques

- Attributs toujours `private`
- Getters/setters `public`
- Valider dans les setters
- Nommer correctement

✎ Astuce IDE

La plupart des IDE (Eclipse, IntelliJ, NetBeans) peuvent **générer automatiquement** les getters et setters !

Le Constructeur

Définition

Un constructeur est une méthode spéciale qui :

- Porte le **même nom** que la classe
- N'a **pas de type de retour** (même pas `void`)
- Est appelé lors de la création d'un objet avec `new`

```java
public class Etudiant {
    private String nom;
    private int age;
    // Constructeur
    public Etudiant(String nom, int age) {
        this.nom = nom;
        this.age = age;
    }
}
// Utilisation
Etudiant e = new Etudiant("Ahmed", 20);
```

Constructeur par Défaut

Sans constructeur défini :

```java
public class Etudiant {
    private String nom;
    private int age;
}

// Java cree un constructeur
// par defaut sans parametre
Etudiant e = new Etudiant();
```

Avec constructeur défini :

```java
public class Etudiant {
    private String nom;
    private int age;

    public Etudiant(String nom,
            int age) {
        this.nom = nom;
        this.age = age;
    }
}

// Etudiant e = new Etudiant();
// ERREUR !
```

Attention

Dès qu'un constructeur est défini, le constructeur par défaut n'est plus généré automatiquement !

Surcharge de Constructeurs

```java
public class Etudiant {
    private String nom;
    private String prenom;
    private double moyenne;
    // Constructeur complet
    public Etudiant(String nom, String prenom, double moyenne) {
        this.nom = nom;
        this.prenom = prenom;
        this.moyenne = moyenne;
    }
    // Constructeur avec moyenne par defaut
    public Etudiant(String nom, String prenom) {
        this(nom, prenom, 0.0); // Appel a l'autre constructeur
    }
    // Constructeur par defaut
    public Etudiant() {
        this("Inconnu", "Inconnu", 0.0);
    }
}
```

Le Mot-clé this

Utilisations de this

1. Référencer l'objet courant
2. Distinguer attribut et paramètre de même nom
3. Appeler un autre constructeur

```java
public class Etudiant {
    private double moyenne;
    public Etudiant(double moyenne) {
        this.moyenne = moyenne; // this.moyenne = attribut
    }
    public void comparer(Etudiant autre) {
        if (this.moyenne > autre.moyenne) {
            System.out.println("Cet etudiant a une meilleure moyenne");
        }
    }
}
```

Création d'Objets

```java
// Declaration + Instanciation
Etudiant e1 = new Etudiant("Ben Ali", "Ahmed", 20, 14.5);

// Declaration separee
Etudiant e2;
e2 = new Etudiant("Gharbi", "Marwa", 21, 16.0);

// Plusieurs objets dans un tableau
Etudiant[] etudiants = new Etudiant[3];
etudiants[0] = new Etudiant("Jebali", "Fatma", 19, 15.5);
etudiants[1] = new Etudiant("Trabelsi", "Mohamed", 22, 12.0);
etudiants[2] = new Etudiant("Bouazizi", "Sara", 20, 17.5);
```

e1
nom : "Ben Ali"
prenom : "Ahmed"
age : 20

Accès aux Attributs et Méthodes

Syntaxe

`nomObjet.attribut` ou `nomObjet.methode()`

```java
public class TestEtudiant {
    public static void main(String[] args) {
        Etudiant etudiant = new Etudiant("Ben Ali", "Ahmed", 20, 14.5);

        // Acces aux methodes
        etudiant.afficher();

        // Avec getters/setters
        etudiant.setMoyenne(15.0);
        double moy = etudiant.getMoyenne();

        System.out.println("Nouvelle moyenne: " + moy);
    }
}
```

La Méthode toString()

Définition

Méthode héritée de la classe `Object` qui retourne une représentation textuelle de l'objet.

```java
public class Etudiant {
    private String nom;
    private String prenom;
    private int age;
    private double moyenne;
    @Override
    public String toString() {
        return "Etudiant{nom='" + nom + "', prenom='" + prenom
                + "', age=" + age + ", moyenne=" + moyenne + "}";
    }
}
// Utilisation
Etudiant e = new Etudiant("Ben Ali", "Ahmed", 20, 14.5);
System.out.println(e); // Appelle automatiquement toString()
// Affiche: Etudiant{nom='Ben Ali', prenom='Ahmed', age=20, moyenne=14.5}
```

null et NullPointerException

Le mot-clé `null`

`null` signifie qu'une variable référence ne pointe vers **aucun objet** en mémoire.

```
Etudiant e1 = new Etudiant("Ahmed", 20); // e1 pointe vers un objet
Etudiant e2 = null;                      // e2 ne pointe vers rien
System.out.println(e1.getNom()); // OK : "Ahmed"
System.out.println(e2.getNom()); // ERREUR a l'execution !
// Exception : java.lang.NullPointerException
```

Attention

Le compilateur ne détecte **pas** cette erreur ! Elle survient uniquement à l'**exécution**.

✓ Bonne pratique : toujours vérifier avant d'utiliser

```
if (e2 != null) {
    e2.afficher();
} else {
    System.out.println("L'etudiant n'existe pas");
}
```

Attributs d'Instance vs Attributs de Classe

- **Attribut d'instance :**
  - Propre à chaque objet
  - Valeur différente par instance
  - Accès via l'objet

```
class Etudiant {
    String nom; // instance
}
Etudiant e = new Etudiant();
e.nom = "Ahmed";
```

- **Attribut de classe** (`static`) :
  - Partagé par tous les objets
  - Une seule valeur pour la classe
  - Accès via le nom de la classe

```
class Etudiant {
    static int nbrEtudiants;
}
Etudiant.nbrEtudiants++;
```

Exemple : Compteur d'Instances

```java
public class Etudiant {
    private String nom;
    private static int nbrEtudiants = 0; // Attribut static

    public Etudiant(String nom) {
        this.nom = nom;
        nbrEtudiants++; // Incremente a chaque creation
    }

    public static int getNbrEtudiants() {
        return nbrEtudiants;
    }
}

// Utilisation
Etudiant e1 = new Etudiant("Ahmed");
Etudiant e2 = new Etudiant("Marwa");
Etudiant e3 = new Etudiant("Fatma");

System.out.println(Etudiant.getNbrEtudiants()); // Affiche: 3
```

Méthodes de Classe (Static)

```java
public class Etudiant {
    private String nom;
    private double moyenne;

    // Methode static pour comparer deux moyennes
    public static Etudiant getMeilleur(Etudiant e1, Etudiant e2) {
        if (e1.moyenne > e2.moyenne) {
            return e1;
        } else {
            return e2;
        }
    }
}

// Utilisation - pas besoin de creer un objet supplementaire
Etudiant e1 = new Etudiant("Ben Ali", "Ahmed", 20, 14.5);
Etudiant e2 = new Etudiant("Gharbi", "Marwa", 21, 16.0);
Etudiant meilleur = Etudiant.getMeilleur(e1, e2);
```

Règles des Méthodes Static

✓ Peuvent accéder à :

- Variables `static`
- Méthodes `static`

✗ Ne peuvent PAS accéder à :

- Variables d'instance
- Méthodes d'instance
- Le mot-clé `this`

	static	**instance**
Méthode d'instance	✓	✓
Méthode static	✓	

Le Bloc Statique (Static Initializer)

Caractéristiques

- Exécuté une seule fois au chargement de la classe
- Exécuté **avant** le `main`
- N'est pas exécuté à chaque instanciation
- N'a pas accès à `this`

```java
public class Etudiant {
    private static String ecole;
    private static int anneeUniversitaire;
    static {
        System.out.println("Initialisation de la classe Etudiant...");
        ecole = "Polytechnique Sousse";
        anneeUniversitaire = 2025;
    }
    public static void main(String[] args) {
        System.out.println("Ecole: " + ecole);
    }
}
// Affiche: Initialisation... puis Ecole: Polytechnique Sousse
```

Le Mot-clé final

■ **Variable** `final` :

- Valeur **non modifiable** après initialisation
- Convention : `MAJUSCULES`

```
final int MAX_AGE = 120;
MAX_AGE = 130; // ERREUR !
```

★ **Constante** `static final` :

- Partagée par toutes les instances
- Valeur fixe pour la classe

```
class Etudiant {
    static final String ECOLE =
        "Polytechnique Sousse";
    static final double
        MOY_MIN = 10.0;
}
```

Résumé

Déclaration	Signification
`final int x = 5`	Variable locale constante
`private final String nom`	Attribut initialisé une seule fois
`static final double PI`	Constante de classe

Qu'est-ce qu'un Package ?

Définition

Un package est un mécanisme d'organisation des classes Java en groupes logiques, similaire à des dossiers dans un système de fichiers.

✔ **Avantages :**

- Organisation du code
- Éviter les conflits de noms
- Contrôle d'accès (visibilité)
- Réutilisation facilitée

☞ **Packages Java courants :**

- `java.lang` – classes de base
- `java.util` – utilitaires
- `java.io` – entrées/sorties
- `java.math` – mathématiques

Note

Le package `java.lang` est importé automatiquement dans tous les programmes Java.

Déclaration d'un Package

Syntaxe

```
package nom.du.package;
```

```java
package tn.polytech.gestion;
public class Etudiant {
    private String nom;
    private double moyenne;
    public Etudiant(String nom, double moyenne) {
        this.nom = nom;
        this.moyenne = moyenne;
    }
}
```

Règles importantes

- La déclaration package doit être la première instruction du fichier
- Le fichier doit être dans un dossier correspondant au package
- Convention : tout en minuscules, séparé par des points

Convention de Nommage des Packages

Convention standard

`domaine_inverse.organisation.projet.module`

✎ **Exemples :**

Organisation	Exemple de package
Polytechnique Sousse	`tn.polytech.gestion.etudiant`
Google	`com.google.maps.api`
Oracle	`com.oracle.database`
Projet personnel	`tn.nom.projet.module`

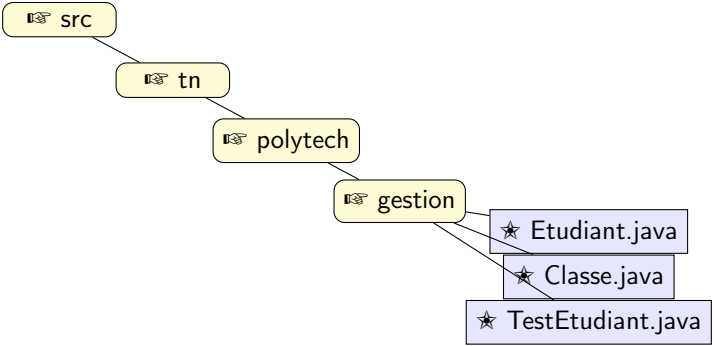
✓ Bonnes pratiques :

- Tout en minuscules
- Utiliser le domaine inversé
- Noms significatifs

✗ À éviter :

- Majuscules
- Caractères spéciaux
- Noms trop génériques

Structure d'un Projet avec Packages




```
package tn.polytech.gestion
```

Correspondance

Le package `tn.polytech.gestion` correspond à la structure de dossiers : `src/tn/polytech/gestion/`

L'Instruction import

Définition

L'instruction `import` permet d'utiliser des classes d'autres packages sans écrire leur nom complet.

Sans import (nom complet) :

```
public class Test {
    public static void main(
            String[] args) {
        java.util.Scanner sc =
            new java.util.Scanner(
                System.in);
        java.util.ArrayList<String>
            liste =
            new java.util.ArrayList<>();
    }
}
```

Avec import :

```
import java.util.Scanner;
import java.util.ArrayList;

public class Test {
    public static void main(
            String[] args) {
        Scanner sc =
            new Scanner(System.in);
        ArrayList<String> liste =
            new ArrayList<>();
    }
}
```

Types d'Imports

➤ Import simple :

```java
// Importe UNE classe
import java.util.Scanner;
import java.util.ArrayList;

// Utilisation
Scanner sc = new Scanner(System.in);
```

☞ Import avec joker (*) :

```java
// Importe TOUTES les classes
import java.util.*;

// Scanner, ArrayList, List...
// tous disponibles
```

✦ Import statique :

```java
// Importe membres statiques
import static java.lang.Math.PI;
import static java.lang.Math.sqrt;

// Utilisation directe
double surface = PI * r * r;
double racine = sqrt(16);
```

★ Import statique (*) :

```java
import static java.lang.Math.*;

double x = sin(PI/2) + cos(0);
```

Recommandation

Préférez les imports simples pour une meilleure lisibilité du code.

Exemple Complet : Organisation avec Packages

Fichier : tn/polytech/gestion/Etudiant.java

```java
package tn.polytech.gestion;
public class Etudiant {
    private String nom;
    private double moyenne;
    public Etudiant(String nom, double moyenne) {
        this.nom = nom;
        this.moyenne = moyenne;
    }
    public void afficher() {
        System.out.println(nom + " - Moyenne: " + moyenne);
    }
}
```

Fichier : tn/polytech/test/TestEtudiant.java

```java
package tn.polytech.test;
import java.util.Scanner;
import tn.polytech.gestion.Etudiant; // Import de notre classe
public class TestEtudiant {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Etudiant e = new Etudiant("Ahmed", 15.5);
        e.afficher();
    }
```

Récapitulatif : Packages et Imports

Concept	Description
`package`	Regroupe les classes en modules logiques
`import`	Permet d'utiliser des classes d'autres packages
`import static`	Permet d'utiliser des membres statiques directement
`java.lang`	Package importé automatiquement
`import pkg.*`	Importe toutes les classes d'un package

✔ Ordre des déclarations dans un fichier Java

1. `package` (optionnel, une seule fois)
2. `import` (optionnel, autant que nécessaire)
3. Déclaration de la classe

Introduction à Scanner

Importation

```
import java.util.Scanner;
```

```
Scanner sc = new Scanner(System.in);
```

Méthodes principales :

Méthode	Type retourné
`nextInt()`	`int`
`nextLong()`	`long`
`nextDouble()`	`double`
`nextLine()`	`String` (ligne entière)
`next()`	`String` (un mot)
`nextLine().charAt(0)`	`char`

Exemple : Saisie d'un Etudiant

```java
import java.util.Scanner;

public class SaisieEtudiant {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Nom de l'etudiant : ");
        String nom = sc.nextLine();
        System.out.print("Prenom de l'etudiant : ");
        String prenom = sc.nextLine();
        System.out.print("Age : ");
        int age = sc.nextInt();
        System.out.print("Moyenne : ");
        double moyenne = sc.nextDouble();
        Etudiant e = new Etudiant(nom, prenom, age, moyenne);
        e.afficher();
        sc.close();
    }
```

Piège classique : nextInt() puis nextLine()

Problème fréquent

Après `nextInt()` ou `nextDouble()`, le caractère `\n` reste dans le buffer.

✗ Code incorrect :

```
System.out.print("Age : ");
int age = sc.nextInt();
System.out.print("Nom : ");
String nom = sc.nextLine();
// nom est vide ! Le \n
// restant est lu directement
```

✓ Code correct :

```
System.out.print("Age : ");
int age = sc.nextInt();
sc.nextLine(); // Vider le buffer
System.out.print("Nom : ");
String nom = sc.nextLine();
// nom contient bien la saisie
```

✎ Règle

Toujours ajouter un `sc.nextLine()` de vidage après `nextInt()`, `nextDouble()`, etc.

Création de Chaînes

Package

`java.lang.String` – importé automatiquement

```java
// Methode 1 : Litterale (recommandee)
String s1 = "Ahmed";
String s11 = "Ahmed";
System.out.println(s1 == s11); // true (meme objet)

// Methode 2 : Avec new
String s2 = new String("Ahmed");
System.out.println(s1 == s2); // false (objets differents)
System.out.println(s1.equals(s2)); // true (meme contenu)
```

Important

La classe `String` est **immuable** : une fois créée, une chaîne ne peut pas être modifiée.

Comparaison d'Objets : == vs equals()

== compare les références

Vérifie si deux variables pointent vers le **même objet** en mémoire.

equals() compare le contenu

Vérifie si deux objets ont les **mêmes valeurs** d'attributs.

```java
Etudiant e1 = new Etudiant("Ahmed", 20);
Etudiant e2 = new Etudiant("Ahmed", 20);
Etudiant e3 = e1;

System.out.println(e1 == e2);       // false (objets differents)
System.out.println(e1 == e3);       // true  (meme reference)
System.out.println(e1.equals(e2)); // false* par defaut !
```

Attention

Par défaut, `equals()` se comporte comme ==. Il faut **redéfinir** `equals()` dans votre classe pour comparer le contenu !

Méthodes de la Classe String

Méthode	Description
`s.length()`	Longueur de la chaîne
`s.equals(ch)`	Comparaison de contenu
`s.charAt(i)`	Caractère à la position i
`s.indexOf(c)`	Position du caractère c
`s.substring(d)`	Sous-chaîne depuis d
`s.substring(d, f)`	Sous-chaîne de d à f-1
`s.toUpperCase()`	En majuscules
`s.toLowerCase()`	En minuscules
`s.trim()`	Supprimer espaces début/fin

```java
String nom = "Ben Ali Ahmed";
System.out.println(nom.length());       // 13
System.out.println(nom.charAt(0));      // 'B'
System.out.println(nom.substring(8));   // "Ahmed"
```

Concaténation et StringBuilder

Avec + (crée des objets intermédiaires) :

```
String prenom = "Ahmed";
String nom = "Ben Ali";
String complet = prenom + " " + nom;
// Cree des chaines intermediaires
```

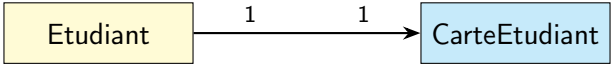
Avec `StringBuilder` (efficace) :

```
StringBuilder sb = new StringBuilder();
sb.append("Ahmed");
sb.append(" ");
sb.append("Ben Ali");
String complet = sb.toString();
```

StringBuilder

Classe mutable permettant de modifier une chaîne sans créer de nouveaux objets. À utiliser pour les concaténations dans les boucles.

Association One-to-One Unidirectionnelle

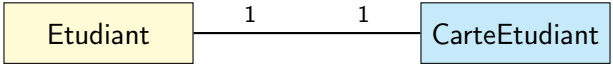



```java
class Etudiant {
    private CarteEtudiant carte;

    public void setCarte(
            CarteEtudiant c) {
        this.carte = c;
    }
    public CarteEtudiant getCarte() {
        return carte;
    }
}
```

```java
class CarteEtudiant {
    private String numero;
    private Date dateExpiration;
    // Pas de reference
    // vers Etudiant
}
```

Association One-to-One Bidirectionnelle



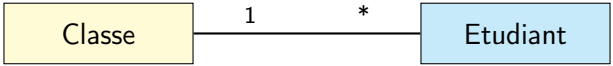

```java
class Etudiant {
    private CarteEtudiant carte;

    public void setCarte(
            CarteEtudiant c) {
        this.carte = c;
    }
    public CarteEtudiant getCarte() {
        return carte;
    }
}
```

```java
class CarteEtudiant {
    private Etudiant etudiant;

    public void setEtudiant(
            Etudiant e) {
        this.etudiant = e;
    }
    public Etudiant getEtudiant() {
        return etudiant;
    }
}
```

Association One-to-Many



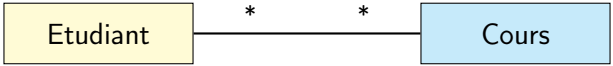

```java
class Classe {
    private Etudiant[] etudiants;

    public void setEtudiants(
            Etudiant[] e) {
        this.etudiants = e;
    }
    public Etudiant[] getEtudiants() {
        return etudiants;
    }
}
```

```java
class Etudiant {
    private Classe classe;

    public void setClasse(
            Classe c) {
        this.classe = c;
    }
    public Classe getClasse() {
        return classe;
    }
}
```

Association Many-to-Many




```java
class Etudiant {
    private Cours[] cours;

    public void setCours(
            Cours[] c) {
        this.cours = c;
    }
    public Cours[] getCours() {
        return cours;
    }
}
```

```java
class Cours {
    private Etudiant[] etudiants;

    public void setEtudiants(
            Etudiant[] e) {
        this.etudiants = e;
    }
    public Etudiant[] getEtudiants() {
        return etudiants;
    }
}
```

QCM 1 – Encapsulation

Question

Quelle option définit une classe bien encapsulée ?

a) `class A { public String nom; }`
b) `class B { String nom; public String getNom() { return nom; } }`
c) `class C { protected String nom; }`
d) Aucune des réponses ci-dessus

Correction QCM 1

Bonne réponse : d) Aucune des réponses ci-dessus

Une classe bien encapsulée nécessite :

- Variables d'instance `private`
- Méthodes publiques getter/setter pour contrôler l'accès

Aucune option ne respecte ces deux principes.

Exemple de classe bien encapsulée

```
class BienEncapsulee {
    private String nom;
    public String getNom() { return nom; }
    public void setNom(String n) { this.nom = n; }
}
```

QCM 2 – Passage d'Objets par Référence

Question

Quelle est la sortie du code suivant ?

```java
class Etudiant { public String nom; }
public class Test {
    public static void main(String[] args) {
        Etudiant e = new Etudiant();
        e.nom = "Ahmed";
        autreMethode(e);
        System.out.println(e.nom);
        modifierNom(e);
        System.out.println(e.nom);
    }
    static void modifierNom(Etudiant e) { e.nom = "Marwa"; }
    static void autreMethode(Etudiant e) {
        e = new Etudiant(); e.nom = "Fatma";
    }
}
```

a) Fatma / Marwa

b) Ahmed / Marwa

c) Ahmed / Ahmed

Correction QCM 2

Bonne réponse : b) Ahmed / Marwa

- `autreMethode(e)` : Crée un nouvel objet localement, l'objet original n'est pas modifié
- `System.out.println(e.nom)` : Affiche "Ahmed" (inchangé)
- `modifierNom(e)` : Modifie l'attribut de l'objet original
- `System.out.println(e.nom)` : Affiche "Marwa"

Principe clé

En Java, les objets sont passés par **valeur de référence** :

- Modifier les attributs → affecte l'objet original
- Réassigner la référence (`e = new...`) → n'affecte PAS l'original

QCM 3 – Passage de Primitifs par Valeur

Question

Quelle est la sortie du code suivant ?

```java
public class Test {
    public static void main(String[] args) {
        int note = 10;
        autreMethode(note);
        System.out.println(note);
        incrementer(note);
        System.out.println(note);
    }
    static void incrementer(int val) { ++val; System.out.println(val); }
    static void autreMethode(int val) { val = 20;
    System.out.println(val); }
}
```

a) 20 / 20 / 11 / 11
b) 20 / 10 / 11 / 11
c) 20 / 10 / 11 / 10

Correction QCM 3

Bonne réponse : c) 20 / 10 / 11 / 10

- `autreMethode(note)` : val = 20 localement, affiche 20
- `System.out.println(note)` : note reste 10
- `incrementer(note)` : val = 11 localement, affiche 11
- `System.out.println(note)` : note reste 10

Principe clé

Les types primitifs (`int`, `double`, `boolean`, etc.) sont passés par **valeur**.
Les modifications dans la méthode n'affectent **jamais** la variable originale.

QCM 4 – Modificateurs d'Accès

Question

Quelles lignes génèrent une erreur de compilation ?

```
1  package tn.polytech.model;
2  public class Etudiant {
3      private int id;
4      protected String nom;
5      public static int compteur = 0;
6      public double moyenne;
7      Etudiant(int i, String n) { id = i; nom = n; }
8  }
9  package tn.polytech.test;
10 import tn.polytech.model.*;
11 public class Test {
12     public static void main(String[] args) {
13         System.out.println(Etudiant.compteur); // L5
14         Etudiant e = new Etudiant(1, "Ali");   // L6
15         System.out.println(e.id);              // L7
16         System.out.println(e.moyenne);         // L8
17         System.out.println(e.nom);             // L9
18     }
19 }
```

a) L5, L7, L9
b) L6, L7, L9
c) L7, L9

Correction QCM 4

Bonne réponse : b) L6, L7, L9

- **L5** ✓ : `compteur` est `public static`, accessible partout
- **L6** × : Constructeur package-private, inaccessible depuis un autre package
- **L7** × : `id` est `private`, inaccessible hors de la classe
- **L8** ✓ : `moyenne` est `public`, accessible partout
- **L9** × : `nom` est `protected`, accessible seulement dans le même package ou par héritage

QCM 5 – Blocs Static

Question

Quelle est la sortie du code suivant ?

```java
public class Etudiant {
    public static int compteur = 5;
    static { compteur = 10; }
    public static void afficher() {
        System.out.print("afficher ");
    }
}
public class Test {
    public static void main(String[] args) {
        Etudiant.afficher();
        new Etudiant().afficher();
        System.out.println(Etudiant.compteur);
    }
}
```

a) afficher afficher 5
b) afficher afficher 10
c) afficher 10

Correction QCM 5

Bonne réponse : b) afficher afficher 10

- Chargement de la classe : `compteur = 5`, puis bloc static : `compteur = 10`
- `Etudiant.afficher()` → affiche "afficher "
- `new Etudiant().afficher()` → affiche "afficher " (méthode static appelée sur instance)
- `System.out.println(compteur)` → affiche 10

Principe clé

Les blocs static s'exécutent une seule fois lors du chargement de la classe, **avant** toute utilisation.

QCM 6 – Méthodes Static vs Instance

Question

Combien d'erreurs de compilation contient ce code ?

```
1  public class Etudiant {
2      private String nom;
3      public static void afficher() {
4          System.out.print("afficher ");
5      }
6      public void etudier() {
7          System.out.println("etudier ");
8      }
9      public static void travailler() {
10         afficher();
11         etudier();              // Ligne 12
12     }
13     public static void main(String[] args) {
14         Etudiant e = new Etudiant();
15         e.travailler();
16         Etudiant e2 = null;
17         e2.afficher();
18     }
19 }
```

a) 0 erreur
b) 1 erreur
c) 2 erreurs

Correction QCM 6

Bonne réponse : b) 1 erreur

Erreur à la ligne 12 :

- `travailler()` est `static`
- `etudier()` est une méthode d'instance
- Une méthode `static` ne peut pas appeler directement une méthode d'instance

Pourquoi `e2.afficher()` compile ?

- Bien que e2 soit `null`, `afficher()` est `static`
- Java appelle la méthode de la classe, pas sur l'instance
- C'est légal mais déconseillé (mauvaise pratique)

Exercice de Synthèse : Gestion des Etudiants

Classe Etudiant - Version Complète

Créer une classe avec :

1. **Attributs d'instance** : `id` (int), `nom` (String), `prenom` (String), `age` (int), `moyenne` (double)
2. **Attribut static** : `nbrEtudiants` (int) pour compter le nombre total d'étudiants
3. **Constructeur paramétré** : initialiser tous les attributs et incrémenter `nbrEtudiants`
4. **Méthodes** :
   - `estAdmis()` : retourne `true` si moyenne ≥ 10
   - `getMention()` : retourne la mention selon la moyenne :
     $\geq$16 : Très Bien, $\geq$14 : Bien, $\geq$12 : Assez Bien, $\geq$10 : Passable, $<$10 : Non admis
   - `toString()` : retourne les informations de l'étudiant formatées
   - `afficher()` : affiche le détail de l'étudiant
5. **Méthode static** : `getNbrEtudiants()` pour retourner le nombre total
6. **Getters/Setters** : pour tous les attributs

Exercice de Synthèse : Gestion des Etudiants (suite)

Classe TestEtudiant (Classe exécutable)

Dans la méthode `main` :

1. Utiliser `Scanner` pour saisir les informations de 3 étudiants :
   - ID
   - Nom et Prénom
   - Age
   - Moyenne
2. Stocker les étudiants dans un tableau de type `Etudiant[]`
3. Pour chaque étudiant créé, afficher ses informations avec `toString()`
4. Effectuer les opérations suivantes :
   - Afficher si chaque étudiant est admis ou non
   - Afficher la mention de chaque étudiant
   - Modifier la moyenne du premier étudiant (+2 points)
5. Afficher le nombre total d'étudiants créés avec `getNbrEtudiants()`

Exercice de Synthèse : Manipulation de String

Questions sur la classe String

Dans la classe `TestEtudiant`, ajouter les traitements suivants :

1. Demander à l'utilisateur de saisir le nom d'un étudiant à rechercher
2. Convertir le nom saisi en majuscules avec `toUpperCase()`
3. Parcourir le tableau des étudiants et comparer les noms avec `equals()` (ignorer la casse avec `equalsIgnoreCase()`)
4. Si l'étudiant est trouvé :
   - Afficher le nom complet de l'étudiant
   - Afficher la longueur du nom avec `length()`
   - Extraire et afficher les 3 premiers caractères avec `substring(0, 3)`
   - Vérifier si le nom contient "ben" avec `indexOf("ben")`
5. Si non trouvé, afficher un message d'erreur

Rappel

Utilisez `equalsIgnoreCase()` pour comparer sans tenir compte de la casse !

Exercice Avancé : Classe Classe

Association One-to-Many : Classe — Etudiant

Créer une classe `Classe` avec :

1. **Attributs** :
   - `nomClasse` (String) ex : "1DNI", "2DNI"
   - `etudiants` (Etudiant[]) : tableau d'étudiants
   - `nombreEtudiantsActuels` (int) : nombre d'étudiants inscrits
2. **Constructeur** : `Classe(String nom, int capacite)`
   - Initialise le nom et crée un tableau de taille `capacite`
3. **Méthodes** :
   - `inscrireEtudiant(Etudiant e)` : ajoute un étudiant si capacité non atteinte
   - `supprimerEtudiant(int id)` : supprime un étudiant par son ID
   - `rechercherEtudiant(int id)` : retourne l'étudiant ou null
   - `afficherTousLesEtudiants()` : affiche tous les étudiants
   - `calculerMoyenneClasse()` : retourne la moyenne de la classe
   - `getMeilleurEtudiant()` : retourne l'étudiant avec la meilleure moyenne

Exercice Avancé : Test de la Classe Classe

Classe TestClasse

Dans la méthode `main` :

1. Créer une classe "1DNI" avec une capacité de 30 étudiants
2. Créer et inscrire 5 étudiants avec Scanner ou directement :
   - `Etudiant(1, "Ben Ali", "Ahmed", 20, 14.5)`
   - `Etudiant(2, "Gharbi", "Marwa", 21, 16.0)`
   - `Etudiant(3, "Jebali", "Mohamed", 22, 12.0)`
   - Etc.
3. Afficher tous les étudiants de la classe
4. Rechercher un étudiant par ID (ex : 2) et afficher ses informations
5. Calculer et afficher la moyenne de la classe
6. Trouver et afficher le meilleur étudiant
7. Supprimer un étudiant (ex : ID 3) et réafficher tous les étudiants
8. Tester l'inscription lorsque la capacité est atteinte

Bonus : Fonctionnalités Avancées

Extensions Possibles

1. **Transfert d'étudiant** :
   - Ajouter une méthode `transfererEtudiant(int id, Classe destination)`
   - Vérifier l'existence de l'étudiant et la capacité de la destination
2. **Statistiques** :
   - `getNbrAdmis()` : compte les étudiants avec moyenne ≥ 10
   - `getTauxReussite()` : pourcentage d'étudiants admis
   - `getNbrParMention(String mention)` : compte par mention
3. **Recherche avancée** :
   - `rechercherParNom(String nom)` : recherche par nom (utiliser `contains()`)
   - `getEtudiantsAdmis()` : retourne les étudiants avec moyenne ≥ 10